

Aufgabenbeschreibung

1. Motivation: Mithilfe generativer künstlicher Intelligenz (KI) und Cloud-nativer Entwicklungsplattformen lassen sich voll funktionsfähige Web-Prototypen in kürzester Zeit automatisiert erzeugen. Diese Anwendungen sind standardmäßig für zustandslose Browser-Laufzeiten und cloudbasierte Datenbanken optimiert. Soll ein solcher Prototyp -am Beispiel eines studentischen Organisations-Dashboards zur Verwaltung sensibler akademischer Daten -aus Gründen der Datensouveränität in eine lokale Desktop-Anwendung überführt werden, bietet sich der Einsatz von hybriden Desktop-Frameworks an. Das Sicherheitsrisiko entsteht hierbei nicht zwingend durch fehlerhaften KI-Code, sondern durch den fundamentalen Paradigmenwechsel der Laufzeitumgebung: Web-Schwachstellen, die im Browser isoliert und moderat risikobehaftet sind, eskalieren in einer Desktop-Laufzeit mit lokalem Systemkontext direkt zu kritischen Betriebssystem-Kompromittierungen (Remote Code Execution). Diese Arbeit untersucht die systematische und sichere Transformation solcher KI-generierten Web-Architekturen.
2. Verwandte Arbeiten: Der aktuelle Forschungsstand fokussiert sich primär auf die Produktivität und Code-Qualität von KI-Generatoren oder auf isolierte Sicherheitsrichtlinien für bestehende Desktop-Hybrid-Apps. Was der Literatur jedoch fehlt, ist eine ganzheitliche, framework-agnostische Betrachtung des eigentlichen Transformationsprozesses. Es existieren kaum wissenschaftliche Leitfäden, die beschreiben, wie cloudbasierte, zustandslose Web-Strukturen systematisch in lokale, ereignisbasierte Multi-Prozess-Architekturen überführt werden müssen, wenn die zugrundeliegende Codebasis von einer dynamischen KI-Modell-Pipeline generiert wurde.
3. Ansatz: Die Arbeit folgt der Methodik des *Design Science Research* (DSR). Zunächst wird mittels KI ein funktionaler Web-Prototyp mit Cloud-Datenbank-Anbindung erzeugt, der typische akademische Kernfeatures (wie Terminplanung und Dokumenten-Exporte) umfasst. Im ersten Schritt wird die Cloud-Infrastruktur durch eine vollständig lokale, dateibasierte relationale Datenhaltung (*Local-First*-Prinzip) ersetzt. Anschließend erfolgt die Portierung in die Desktop-Laufzeit. Um die erweiterten Privilegien der Desktop-Umgebung abzusichern, wird ein systematisches Härtingsverfahren implementiert: Die netzwerkbasierte Kommunikation wird durch strikt isolierte Inter-Prozess-Kommunikations-Kanäle (IPC) mittels Preload-Skripten und Context Isolation ersetzt, um die Benutzeroberfläche vollständig vom Betriebssystem zu entkoppeln.
4. Validierung/Auswertung: Die Evaluierung erfolgt zweistufig. Zuerst wird durch ein qualitatives Sicherheits-Audit nachgewiesen, dass potenzielle Angriffsvektoren (wie unvollständige Eingabe- und Pfadvalidierungen bei lokalen Dateioperationen) durch die restriktive IPC-Architektur und Content Security Policies (CSP) effektiv neutralisiert wurden. Ergänzend werden in einer vergleichenden Analyse der durch die Härtung entstandene architektonische Refactoring-Aufwand sowie die Veränderung des lokalen Ressourcen-Footprints (CPU- und Memory-Auslastung durch Prozess-Isolation) kritisch gemessen und diskutiert.